

Syntax Analysis

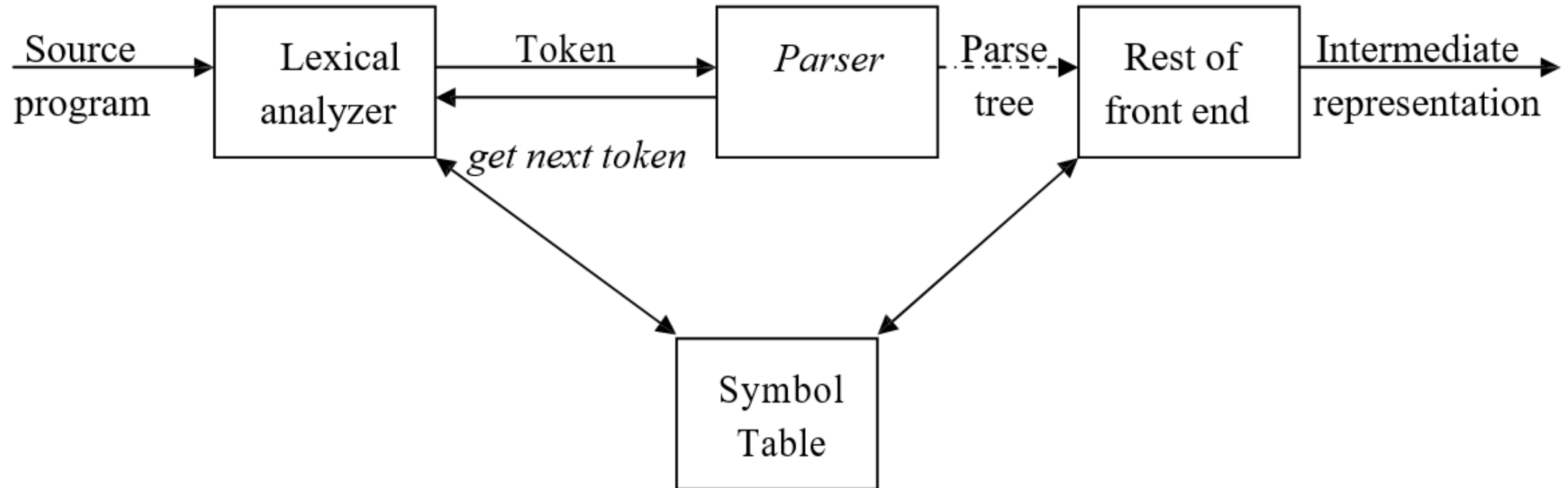
Definition

- Syntax analysis is the second phase of the compiler.
- It gets the input from the tokens and generates a syntax tree or parse tree.

The Role of a parser

- parser or syntactic analyzer obtains a string of tokens from the lexical analyzer and verifies that the string can be generated by the grammar for the source language.
- It reports any syntax errors in the program.
- It also recovers from commonly occurring errors so that it can continue processing its input.

Position of parser in compiler model



Functions of the parser

1. It verifies the structure generated by the tokens based on the grammar.
2. It constructs the parse tree.
3. It reports the errors.
4. It performs error recovery.

Issues

Parser cannot detect errors such as:

1. Variable re-declaration
2. Variable initialization before use.
3. Data type mismatch for an operation.

The above issues are handled by Semantic Analysis phase.

Syntax error handling :

Programs can contain errors at many different levels. For example :

1. Lexical, such as misspelling a keyword.
2. Syntactic, such as an arithmetic expression with unbalanced parentheses.
3. Semantic, such as an operator applied to an incompatible operand.
4. Logical, such as an infinitely recursive call.

CONTEXT-FREE GRAMMARS

- A Context-Free Grammar is a quadruple that consists of **terminals**, **non-terminals**, **start symbol** and **productions**.
- **Terminals** : These are the basic symbols from which strings are formed.
- **Non-Terminals** : These are the syntactic variables that denote a set of strings. These help to define the language generated by the grammar.
- **Start Symbol** : One non-terminal in the grammar is denoted as the “Start-symbol” and the set of strings it denotes is the language defined by the grammar.
- **Productions** : It specifies the manner in which terminals and non-terminals can be combined to form strings. Each production consists of a non-terminal, followed by an arrow, followed by a string of non-terminals and terminals.

Example

- The following grammar defines simple arithmetic expressions:

$\text{expr} \rightarrow \text{expr op expr}$

$\text{expr} \rightarrow (\text{expr})$

$\text{expr} \rightarrow - \text{expr}$

$\text{expr} \rightarrow \text{id}$

$\text{op} \rightarrow +$

$\text{op} \rightarrow -$

$\text{op} \rightarrow *$

$\text{op} \rightarrow /$

$\text{op} \rightarrow \uparrow$

In this grammar,

- $\text{id} + - * / \uparrow ()$ are terminals.
- expr , op are non-terminals.
- expr is the start symbol.
- Each line is a production.

Derivations

- Two basic requirements for a grammar are :
 1. To generate a valid string.
 2. To recognize a valid string.

Derivation is a process that generates a valid string with the help of grammar by replacing the non-terminals on the left with the string on the right side of the production.

Types of derivations

- The two types of derivation are:
 1. Left most derivation
 2. Right most derivation.
- In leftmost derivations, the leftmost non-terminal in each sentinel is always chosen first for replacement.
- In rightmost derivations, the rightmost non-terminal in each sentinel is always chosen first for replacement.

Sentinels:

Given a grammar G with start symbol S , if $S \rightarrow \alpha$, where α may contain non-terminals or terminals, then α is called the sentinel form of G .

Example:

- Given grammar $G : E \rightarrow E+E \mid E^*E \mid (E) \mid - E \mid id$
- Sentence to be derived : $-(id+id)$

LEFTMOST DERIVATION

$E \rightarrow - E$
 $E \rightarrow - (E)$
 $E \rightarrow - (E+E)$
 $E \rightarrow - (id+E)$
 $E \rightarrow - (id+id)$

RIGHTMOST DERIVATION

$E \rightarrow - E$
 $E \rightarrow - (E)$
 $E \rightarrow - (E+E)$
 $E \rightarrow - (E+id)$
 $E \rightarrow - (id+id)$

String that appear in leftmost derivation are called left sentinel forms.

String that appear in rightmost derivation are called right sentinel forms

Ambiguity

- A grammar that produces more than one parse for some sentence is said to be **ambiguous grammar**.

Given grammar

$G : E \rightarrow E + E \mid E * E \mid (E) \mid - E \mid id$

$E \rightarrow E + E$

$E \rightarrow id + E$

$E \rightarrow id + E * E$

$E \rightarrow id + id * E$

$E \rightarrow id + id * id$

$E \rightarrow E * E$

$E \rightarrow E + E * E$

$E \rightarrow id + E * E$

$E \rightarrow id + id * E$

$E \rightarrow id + id * id$

The sentence **id+id*id** has the following two distinct leftmost derivations:

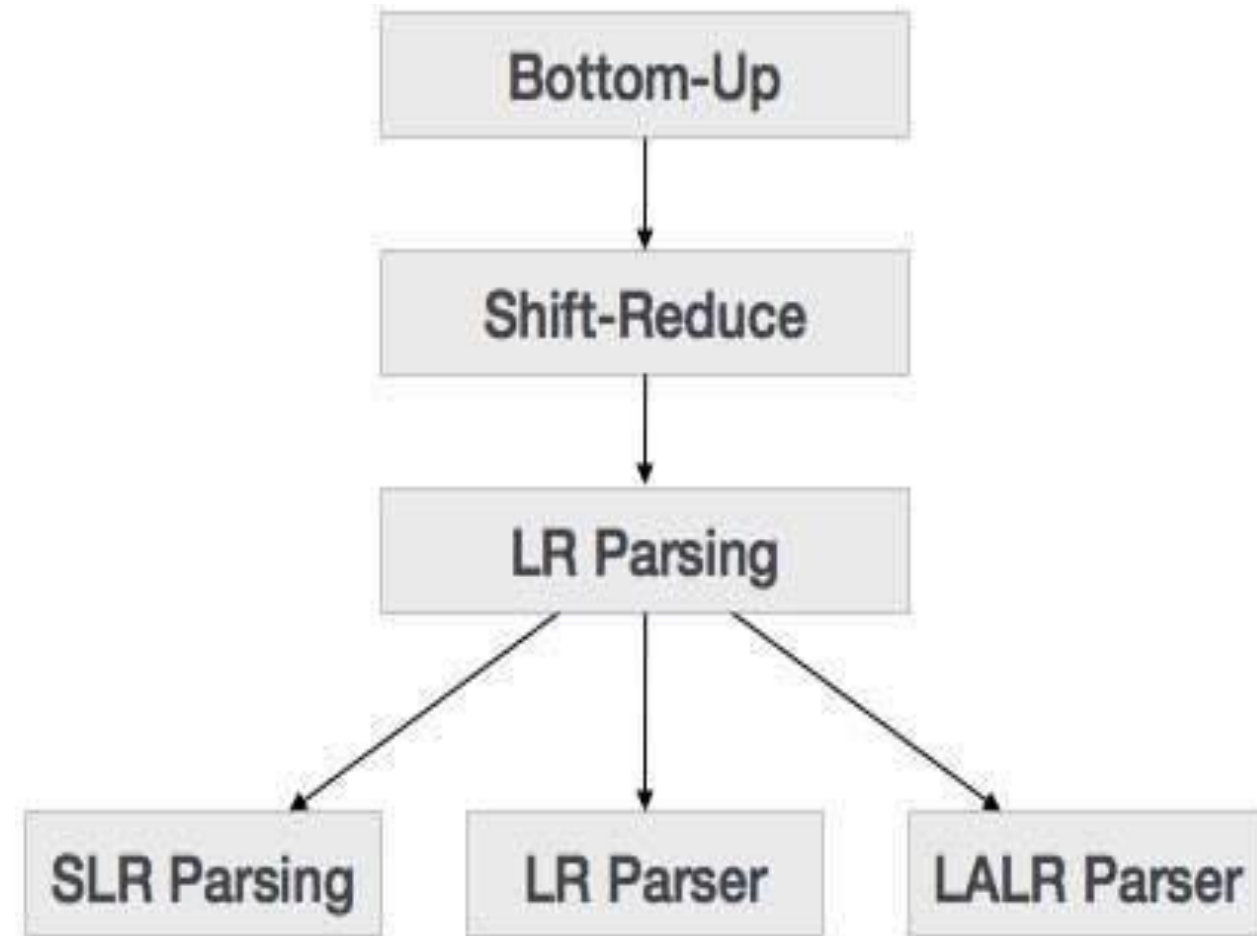
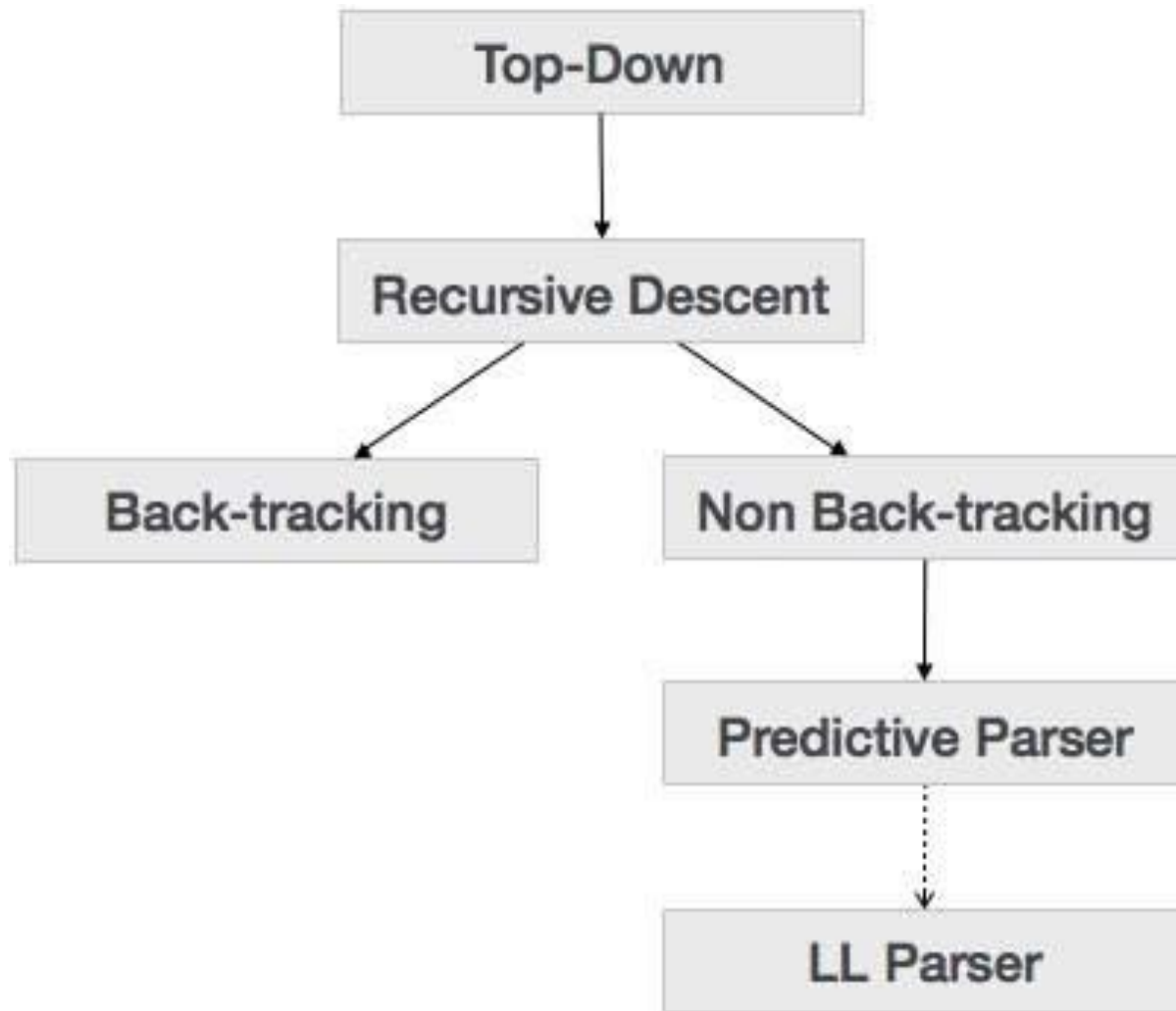
The two corresponding parse trees are :

E			
E	+	E	
id		E	* E

E			
E		*	E
E	+	E	id

Parsing

- Given a stream of input tokens, parsing involves the process of reducing them to a nonterminal.
- Parsing can be either top-down or bottom-up.
- **Top-down parsing** involves generating the string starting from the first non-terminal and repeatedly applying production rules.
- **Bottom-up parsing** involves repeatedly rewriting the input string until it ends up in the first nonterminal of the grammar.



Top-Down Parsing

- The parse tree is created top to bottom.
- Top-down parser
 - Recursive-Descent Parsing
 - Predictive Parsing

Recursive-Descent Parsing

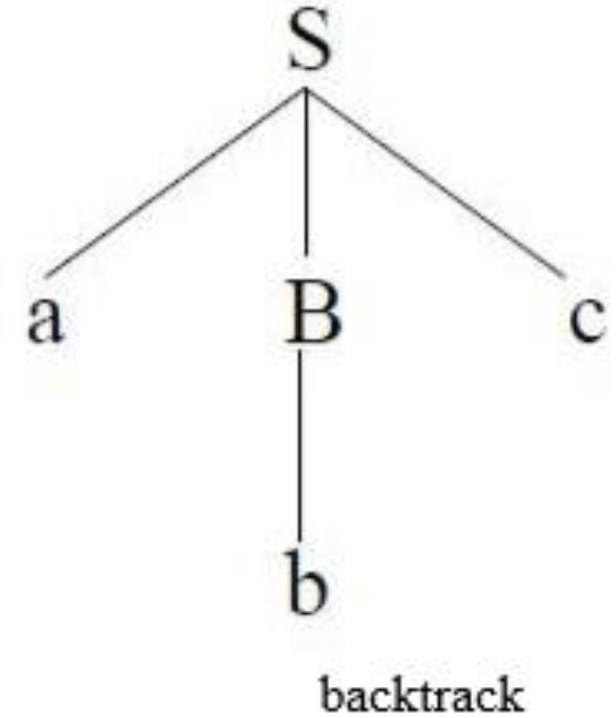
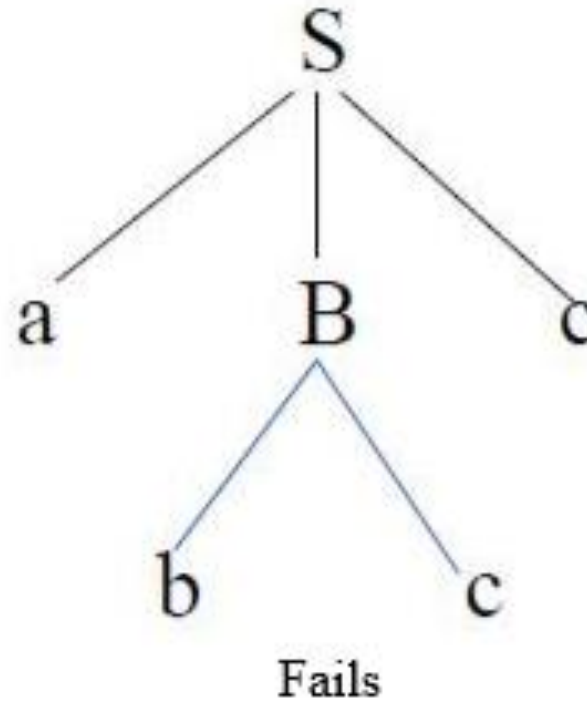
- Backtracking is needed (If a choice of a production rule does not work, we backtrack to try other alternatives.)
- It is a general parsing technique, but not widely used.
- Not efficient
- It tries to find the left-most derivation

Example

$S \rightarrow aBc$

$B \rightarrow bc|b$

Input: abc



Method: let input $w = abc$, initially create the tree of single node S . The left most node a match the first symbol of w , so advance the pointer to b and consider the next leaf B . Then expand B using first choice bc . There is match for b and c , and advanced to the leaf symbol c of S , but there is no match in input, report failure and go back to B to find another alternative b that produce match.

Eliminating Left Recursion

- A grammar is said to be *left recursive* if it has a non-terminal A such that there is a derivation $A \rightarrow A\alpha$ for some string α .
- Top-down parsing methods cannot handle left-recursive grammars. Hence, left recursion can be eliminated as follows:
- If there is a production $A \rightarrow A\alpha \mid \beta$ it can be replaced with a sequence

$$A \rightarrow \beta A'$$

$$A' \rightarrow \alpha A' \mid \epsilon$$

without changing the set of strings derivable from A .

Immediate Left - Recursion

$$\begin{array}{l} A \rightarrow A\alpha \mid \beta \\ \quad \downarrow \text{Eliminate immediate left recursion} \\ A \rightarrow \beta A' \\ A' \rightarrow \alpha A' \mid \epsilon \end{array}$$

In general,

$$A \rightarrow A \alpha_1 \mid \dots \mid A \alpha_m \mid \beta_1 \mid \dots \mid \beta_n \quad \text{where } \beta_1 \dots \beta_n \text{ do not start with } A$$

$\Downarrow$ eliminate immediate left recursion

$$A \rightarrow \beta_1 A' \mid \dots \mid \beta_n A'$$

$$A' \rightarrow \alpha_1 A' \mid \dots \mid \alpha_m A' \mid \epsilon \quad \text{an equivalent grammar}$$

Example :

- Consider the following grammar for arithmetic expressions:

$$E \rightarrow E+T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow (E) \mid \text{id}$$

- First eliminate the left recursion for E as

$$E \rightarrow TE'$$

$$E' \rightarrow +TE' \mid \epsilon$$

- Then eliminate for T as

$$T \rightarrow FT' \quad T' \rightarrow *FT' \mid \epsilon$$

Thus the obtained grammar after eliminating left recursion is

$$E \rightarrow TE'$$

$$E' \rightarrow +TE' \mid \varepsilon$$

$$T \rightarrow FT'$$

$$T' \rightarrow *FT' \mid \varepsilon$$

$$F \rightarrow (E) \mid \text{id}$$

Non-Immediate Left-Recursion

- By just eliminating the immediate left-recursion, we may not get a grammar which is not left recursive.

$$S \rightarrow Aa \mid b$$

$A \rightarrow Sc \mid d$ This grammar is not immediately left-recursive,
but it is still left-recursive.

$$\underline{S} \Rightarrow Aa \Rightarrow \underline{S}ca \quad \text{or}$$

$$\underline{A} \Rightarrow Sc \Rightarrow \underline{A}ac \quad \text{causes to a left-recursion}$$

So, we have to eliminate all left-recursions from our grammar

Left factoring

- Left factoring is a grammar transformation that is useful for producing a grammar suitable for predictive parsing.
- When it is not clear which of two alternative productions to use to expand a non-terminal A , we can rewrite the A -productions to defer the decision until we have seen enough of the input to make the right choice.
- **If there is any production $A \rightarrow \alpha\beta_1 \mid \alpha\beta_2$, it can be rewritten as**
 $A \rightarrow \alpha A'$
 $A' \rightarrow \beta_1 \mid \beta_2$

- In General:

Replace productions

$$A \rightarrow \alpha \beta_1 \mid \alpha \beta_2 \mid \dots \mid \alpha \beta_n \mid \gamma$$

with

$$A \rightarrow \alpha A' \mid \gamma$$

$$A' \rightarrow \beta_1 \mid \beta_2 \mid \dots \mid \beta_n$$

Hint: taking α common from the each production.

Example

- Consider the grammar, G :

$$S \rightarrow iEtS \mid iEtSeS \mid a$$

$$E \rightarrow b$$

Left factoring, this grammar becomes

$$S \rightarrow iEtSS' \mid a$$

$$S' \rightarrow eS \mid \epsilon$$

$$E \rightarrow b$$

Top-down parser

1. Recursive-Descent Parsing

- Backtracking is needed (If a choice of a production rule does not work, we backtrack to try other alternatives.)
- It is a general parsing technique, but not widely used.
- Not efficient

2. Predictive Parsing

- no backtracking
- Efficient
- Use LL (Left-to-right, Leftmost derivation) methods
- needs a special form of grammars (LL(1) grammars).
- Recursive Predictive Parsing is a special form of Recursive Descent parsing without backtracking.
- Non-Recursive (Table Driven) Predictive Parser is also known as LL(1) parser.

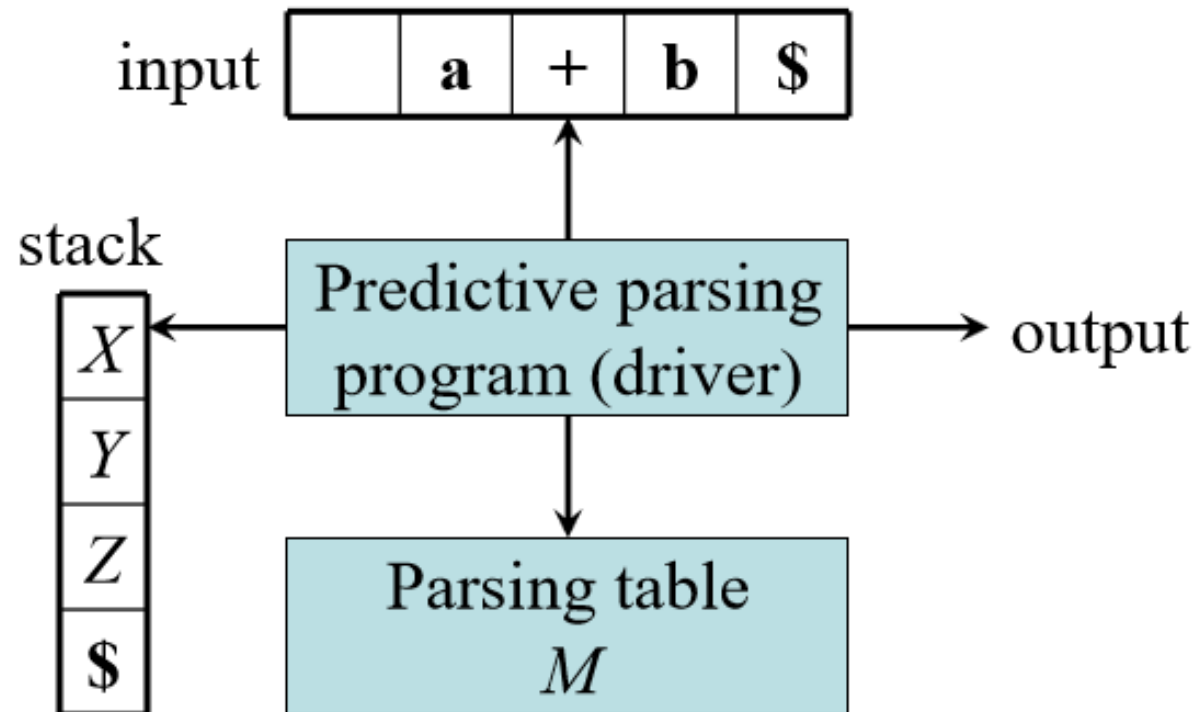
Predictive Parsing

- A predictive parser tries to predict which production produces the least chances of a backtracking and infinite looping.
- When re-writing a non-terminal in a derivation step, a predictive parser can uniquely choose a production rule by just looking the current symbol in the input string.
- Example
- `Stmt` $\rightarrow$ `if.....` | `while.....` | `begin.....` | `for.....`
- When we are trying to write the non-terminal `stmt`, if the current token is `if` we have to choose first production rule.

- Two variants:
 - Recursive (recursive-descent parsing)
 - Non-recursive (table-driven parsing)

Non-Recursive Predictive Parsing

- Non-Recursive predictive parsing is a table-driven parser.
- Given an LL(1) grammar $G = (N, T, P, S)$ construct a table $M[A,a]$ for $A \in N$, $a \in T$ and use a driver program with a stack.



Non-Recursive Predictive Parsing

input buffer

- our string to be parsed. We will assume that its end is marked with a special symbol \$.

output

- a production rule representing a step of the derivation sequence (left-most derivation) of the string in the input buffer.

stack

- contains the grammar symbols
- at the bottom of the stack, there is a special end marker symbol \$.
- initially the stack contains only the symbol \$ and the starting symbol S.
- when the stack is emptied (ie. only \$ left in the stack), the parsing is completed.

parsing table

- a two-dimensional array $M[A, a]$
- each row is a non-terminal symbol
- each column is a terminal symbol or the special symbol \$
- each entry holds a production rule.

Algorithm

Input : a string w .

Output: if w is in $L(G)$, a leftmost derivation of w ; otherwise error

1. Set ip to the first symbol of input stream

2. Set the stack to $\$S$ where S is the start symbol of the grammar

3. repeat

Let X be the top stack symbol and a be the symbol pointed by ip

If X is a terminal or $\$$ then

if $X = a$ then pop X from the stack and advance ip

else error()

else /* X is a non-terminal */

if $M[X, a] = X \rightarrow Y_1, Y_2, \dots, Y_k$ then

pop X from stack

push $Y_k, Y_{k-1}, \dots, Y_1$ onto stack (with Y_1 on top)

output the production $X \rightarrow Y_1, Y_2, \dots, Y_k$

else error()

4. until $X = \$$ /* stack is empty */

Algorithm

Input : a string w .

Output: if w is in $L(G)$, a leftmost derivation of w ; otherwise error

1. Set ip to the first symbol of input stream

2. Set the stack to $\$S$ where S is the start symbol of the grammar

3. repeat

Let X be the top stack symbol and a be the symbol pointed by ip

If X is a terminal or $\$$ then

if $X = a$ then pop X from the stack and advance ip

else error()

else /* X is a non-terminal */

if $M[X, a] = X \rightarrow Y_1, Y_2, \dots, Y_k$ then

pop X from stack

push $Y_k, Y_{k-1}, \dots, Y_1$ onto stack (with Y_1 on top)

output the production $X \rightarrow Y_1, Y_2, \dots, Y_k$

else error()

4. until $X = \$$ /* stack is empty */

$S \rightarrow aBa$

$B \rightarrow bB \mid \epsilon$

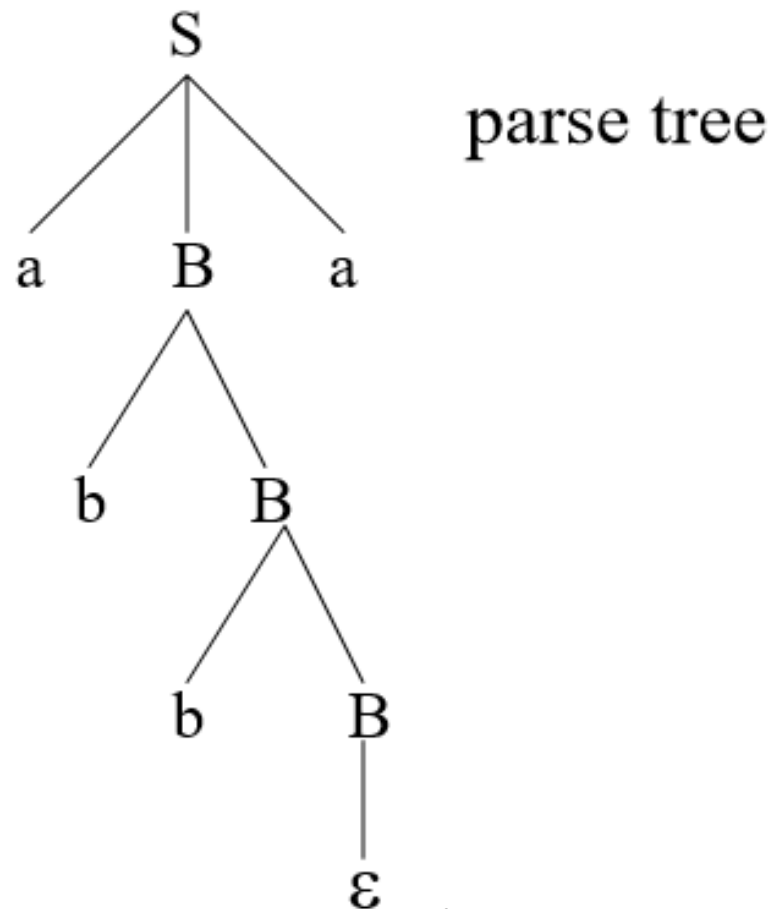
<u>stack</u>	<u>input</u>	<u>output</u>
$\$S$	abba\$	$S \rightarrow aBa$
$\$aBa$	abba\$	
$\$aB$	bba\$	$B \rightarrow bB$
$\$aBb$	bba\$	
$\$aB$	ba\$	$B \rightarrow bB$
$\$aBb$	ba\$	
$\$aB$	a\$	$B \rightarrow \epsilon$
$\$a$	a\$	
$\$$	\$	accept

	a	b	\$
S	$S \rightarrow aBa$		
B	$B \rightarrow \epsilon$	$B \rightarrow bB$	

LL(1) Parsing
Table

Outputs: $S \rightarrow aBa$ $B \rightarrow bB$ $B \rightarrow bB$ $B \rightarrow \varepsilon$

Derivation(left-most): $S \Rightarrow aBa \Rightarrow abBa \Rightarrow abbBa \Rightarrow abba$



Constructing LL(1) Parsing Table

1. Eliminate left recursion from grammar
2. Left factor the grammar

a grammar $\xrightarrow{\substack{\text{eliminate} \\ \text{left recursion}}}$ $\xrightarrow{\substack{\text{left} \\ \text{factor}}}$ a grammar suitable for predictive parsing (a LL(1) grammar)

3. Compute **FIRST** and **FOLLOW** functions

FIRST and FOLLOW

- **FIRST(α)** is a set of the terminal symbols which occur as first symbols in strings derived from α
where α is any string of grammar symbols.
if α derives to ϵ , then ϵ is also in FIRST(α) .
- **FOLLOW(A)** is the set of the terminals which occur immediately after (follow) the *non-terminal* A in the strings derived from the starting symbol.
 - a terminal a is in FOLLOW(A) if $S \Rightarrow \alpha A a \beta$
 - \$ is in FOLLOW(A) if $S \Rightarrow \alpha A$

Compute FIRST

1. If X is a terminal symbol then $\text{FIRST}(X) = \{X\}$
 2. If X is a non-terminal symbol and $X \rightarrow \epsilon$ is a production rule then
$$\text{FIRST}(X) = \text{FIRST}(X) \cup \epsilon.$$
 3. If X is a non-terminal symbol and $X \rightarrow Y_1 Y_2 \dots Y_n$ is a production rule then
 - a. if a terminal a in $\text{FIRST}(Y_1)$ then $\text{FIRST}(X) = \text{FIRST}(X) \cup \text{FIRST}(Y_1)$
 - b. if a terminal a in $\text{FIRST}(Y_i)$ and ϵ is in all $\text{FIRST}(Y_j)$ for $j=1, \dots, i-1$ then
$$\text{FIRST}(X) = \text{FIRST}(X) \cup a.$$
 - c. if ϵ is in all $\text{FIRST}(Y_j)$ for $j=1, \dots, n$ then $\text{FIRST}(X) = \text{FIRST}(X) \cup \epsilon.$
- If X is ϵ then $\text{FIRST}(X) = \{\epsilon\}$
 - If X is $Y_1 Y_2 \dots Y_n$
 - a. if a terminal a in $\text{FIRST}(Y_i)$ and ϵ is in all $\text{FIRST}(Y_j)$ for $j=1, \dots, i-1$ then
$$\text{FIRST}(X) = \text{FIRST}(X) \cup a$$
 - b. if ϵ is in all $\text{FIRST}(Y_j)$ for $j=1, \dots, n$ then $\text{FIRST}(X) = \text{FIRST}(X) \cup \epsilon.$

Compute FIRST: Example

$$E \rightarrow TE'$$

$$E' \rightarrow +TE' \mid \varepsilon$$

$$T \rightarrow FT'$$

$$T' \rightarrow *FT' \mid \varepsilon$$

$$F \rightarrow (E) \mid id$$

$$FIRST(F) = \{ (, id \}$$

$$FIRST(T') = \{ *, \varepsilon \}$$

$$FIRST(T) = \{ (, id \}$$

$$FIRST(E') = \{ +, \varepsilon \}$$

$$FIRST(E) = \{ (, id \}$$

$$FIRST(TE') = \{ (, id \}$$

$$FIRST(+TE') = \{ + \}$$

$$FIRST(\varepsilon) = \{ \varepsilon \}$$

$$FIRST(FT') = \{ (, id \}$$

$$FIRST(*FT') = \{ * \}$$

$$FIRST(\varepsilon) = \{ \varepsilon \}$$

$$FIRST((E)) = \{ (\}$$

$$FIRST(id) = \{ id \}$$

Compute FOLLOW

Apply the following rules until nothing can be added to any FOLLOW set:

1. If S is the start symbol then $\$$ is in $\text{FOLLOW}(S)$
2. if $A \rightarrow \alpha B \beta$ is a production rule then everything in $\text{FIRST}(\beta)$ is placed in $\text{FOLLOW}(B)$ except ϵ
3. If ($A \rightarrow \alpha B$ is a production rule) or ($A \rightarrow \alpha B \beta$ is a production rule and ϵ is in $\text{FIRST}(\beta)$) then everything in $\text{FOLLOW}(A)$ is in $\text{FOLLOW}(B)$.

Compute FOLLOW Example

$$E \rightarrow TE'$$

$$E' \rightarrow +TE' \mid \varepsilon$$

$$T \rightarrow FT'$$

$$T' \rightarrow *FT' \mid \varepsilon$$

$$F \rightarrow (E) \mid \text{id}$$

$$\text{FOLLOW}(E) = \{ \$,) \}$$

$$\text{FOLLOW}(E') = \{ \$,) \}$$

$$\text{FOLLOW}(T) = \{ +,), \$ \}$$

$$\text{FOLLOW}(T') = \{ +,), \$ \}$$

$$\text{FOLLOW}(F) = \{ +, *,), \$ \}$$

Constructing LL(1) Parsing Tables

Algorithm

Input: LL(1) Grammar G

Output: Parsing Table M

for each production rule $A \rightarrow \alpha$ of a grammar G

 for each terminal a in $\text{FIRST}(\alpha)$

 add $A \rightarrow \alpha$ to $M[A,a]$

 If ϵ in $\text{FIRST}(\alpha)$ then

 for each terminal a in $\text{FOLLOW}(A)$

 add $A \rightarrow \alpha$ to $M[A,a]$

 If ϵ in $\text{FIRST}(\alpha)$ and $\$$ in $\text{FOLLOW}(A)$ then

 add $A \rightarrow \alpha$ to $M[A,\$]$

make all other undefined entries of the parsing table M be error

$E \rightarrow TE'$	$\text{FIRST}(TE') = \{ (, \text{id} \}$	$E \rightarrow TE' \text{ into } M[E, (] \text{ and } M[E, \text{id}]$
$E' \rightarrow +TE'$	$\text{FIRST}(+TE') = \{ + \}$	$E' \rightarrow +TE' \text{ into } M[E', +]$
$E' \rightarrow \varepsilon$	$\text{FIRST}(\varepsilon) = \{ \varepsilon \}$ but since ε in $\text{FIRST}(\varepsilon)$ and $\text{FOLLOW}(E') = \{ \$,) \}$	$E' \rightarrow \varepsilon \text{ into } M[E', \$] \text{ and } M[E',)]$
$T \rightarrow FT'$	$\text{FIRST}(FT') = \{ (, \text{id} \}$	$T \rightarrow FT' \text{ into } M[T, (] \text{ and } M[T, \text{id}]$
$T' \rightarrow *FT'$	$\text{FIRST}(*FT') = \{ * \}$	$T' \rightarrow *FT' \text{ into } M[T', *]$
$T' \rightarrow \varepsilon$	$\text{FIRST}(\varepsilon) = \{ \varepsilon \}$ but since ε in $\text{FIRST}(\varepsilon)$ and $\text{FOLLOW}(T') = \{ \$,), + \}$	$T' \rightarrow \varepsilon \text{ into } M[T', \$], M[T',)] \text{ and } M[T', +]$
$F \rightarrow (E)$	$\text{FIRST}((E)) = \{ (\}$	$F \rightarrow (E) \text{ into } M[F, (]$
$F \rightarrow \text{id}$	$\text{FIRST}(\text{id}) = \{ \text{id} \}$	$F \rightarrow \text{id} \text{ into } M[F, \text{id}]$

Predictive parsing table

NON- TERMINAL	id	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \epsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F	$F \rightarrow \text{id}$			$F \rightarrow (E)$		

Stack implementation

stack	Input	Output
\$E	id+id*id \$	
\$E'T	id+id*id \$	$E \rightarrow TE'$
\$E'T'F	id+id*id \$	$T \rightarrow FT'$
\$E'T'id	id+id*id \$	$F \rightarrow id$
\$E'T'	+id*id \$	
\$E'	+id*id \$	$T' \rightarrow \epsilon$
\$E'T+	+id*id \$	$E' \rightarrow +TE'$
\$E'T	id*id \$	
\$E'T'F	id*id \$	$T \rightarrow FT'$
\$E'T'id	id*id \$	$F \rightarrow id$
\$E'T'	*id \$	
\$E'T'F*	*id \$	$T' \rightarrow *FT'$
\$E'T'F	id \$	
\$E'T'id	id \$	$F \rightarrow id$
\$E'T'	\$	
\$E'	\$	$T' \rightarrow \epsilon$
\$	\$	$E' \rightarrow \epsilon$

LL(1) Grammar

- A grammar whose parsing table has no multiply-defined entries is said to be LL(1) grammar.
- *What happen when a parsing table contains multiply defined entries ? – The problem is ambiguity*
- A left recursive, not left factored and ambiguous grammar cannot be a LL(1) grammar (i.e. left recursive, not left factored and ambiguous grammar may have multiply –defined entries in parsing table)
- *There are no general rules by which multiply-defined entries can be made single-valued without affecting the language recognized by a grammar – therefore there should be LL(1) grammar as an input to construct the parsing table*

Example

$$S \rightarrow iEtS \mid iEtSeS \mid a$$
$$E \rightarrow b$$

left factoring, we have

$$S \rightarrow iEtSS' \mid a$$
$$S' \rightarrow eS \mid \varepsilon$$
$$E \rightarrow b$$

- To construct a parsing table, we need FIRST() and FOLLOW() for all the non-terminals.
- $\text{FIRST}(S) = \{ i, a \}$
- $\text{FIRST}(S') = \{ e, \epsilon \}$
- $\text{FIRST}(E) = \{ b \}$
- $\text{FOLLOW}(S) = \{ \$, e \}$
- $\text{FOLLOW}(S') = \{ \$, e \}$
- $\text{FOLLOW}(E) = \{ t \}$

Parsing table

NON- TERMINAL	a	b	e	i	t	\$
S	$S \rightarrow a$			$S \rightarrow iEtSS'$		
S'			$S' \rightarrow eS$ $S' \rightarrow \epsilon$			$S' \rightarrow \epsilon$
E		$E \rightarrow b$				

Since there are more than one production, the grammar is not LL(1) grammar.

Recall

Actions performed in predictive parsing:

1. Shift
2. Reduce
3. Accept
4. Error

Implementation of predictive parser:

1. Elimination of left recursion, left factoring and ambiguous grammar.
2. Construct FIRST() and FOLLOW() for all non-terminals.
3. Construct predictive parsing table.
4. Parse the given input string using stack and parsing table.

Properties of LL(1) Grammars

one input symbol used as a look-head symbol to determine parser action

LL(1) — left most derivation
↑
input scanned from left to right

A grammar G is LL(1) if and only if the following conditions hold for two distinctive production rules $A \rightarrow \alpha$ and $A \rightarrow \beta$

1. Both α and β cannot derive strings starting with same terminals.
2. At most one of α and β can derive to ϵ .
3. If β can derive to ϵ , then α cannot derive to any string starting with a terminal in $\text{FOLLOW}(A)$.

Error Recovery in Predictive Parsing

An error may occur in the predictive parsing (LL(1) parsing)

- if the terminal symbol on the top of stack does not match with the current input symbol.
- if the top of stack is a non-terminal A , the current input symbol is a , and the parsing table entry $M[A,a]$ is empty.

What should the parser do in an error case?

- The parser should be able to give an error message (as much as possible meaningful error message).
- It should be recover from that error case, and it should be able to continue the parsing with the rest of the input.

Assignment

1. Consider the grammar:

$$S \rightarrow (L) \mid a$$
$$L \rightarrow L, S \mid S$$

Derive the parse tree for the input (a,a)

2. What do you mean by left recursive grammar? Remove the left recursion in the following grammar

$$E \rightarrow E+T \mid T$$
$$T \rightarrow T * F \mid F$$
$$F \rightarrow \text{id} \mid (E)$$

3. What do you mean by non immediate left recursion? Remove the left recursion (if any)

$$S \rightarrow Aa \mid b$$
$$A \rightarrow Sc \mid d$$

4. What is predictive parsing? Explain the different component of predictive parsing.

5. Compute the first and follow , then LL(1) Parsing table for following grammar.

$$S \rightarrow aBa$$
$$B \rightarrow bB \mid \epsilon$$

6. Construct the predictive parsing

$$E \rightarrow E+T \mid T$$
$$T \rightarrow T * F \mid F$$
$$F \rightarrow \text{id} \mid (E)$$

Now trace the parsing steps for the input id*id+id

7. What do you mean by LL(1) grammar? Write the properties of LL(1) grammar.

BOTTOM-UP PARSING

- Constructing a parse tree for an input string beginning at the leaves and going towards the root is called bottom-up parsing.
- A general type of bottom-up parser is a **shift-reduce parser**

SHIFT-REDUCE PARSING

- Shift-reduce parsing is a type of bottom-up parsing that attempts to construct a parse tree for an input string beginning at the leaves (the bottom) and working up towards the root (the top).

Example:

Consider the grammar:

$S \rightarrow aABe$

$A \rightarrow Abc \mid b$

$B \rightarrow d$

The sentence to be recognized is *abbcde*.

REDUCTION (LEFTMOST)

abbcde ($A \rightarrow b$)

aAbcde ($A \rightarrow Abc$)

aAde ($B \rightarrow d$)

aABe ($S \rightarrow aABe$)

S

RIGHTMOST DERIVATION

S $\rightarrow$ **aABe**

$\rightarrow$ **aAde**

$\rightarrow$ **aAbcde**

$\rightarrow$ **abbcde**

The reductions trace out the right-most derivation in reverse.

Handles

- A handle of a string is a substring that matches the right side of a production, and whose reduction to the non-terminal on the left side of the production represents one step along the reverse of a rightmost derivation.
- Example 1: Let's take a grammar

$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow (E) \mid \text{id}$

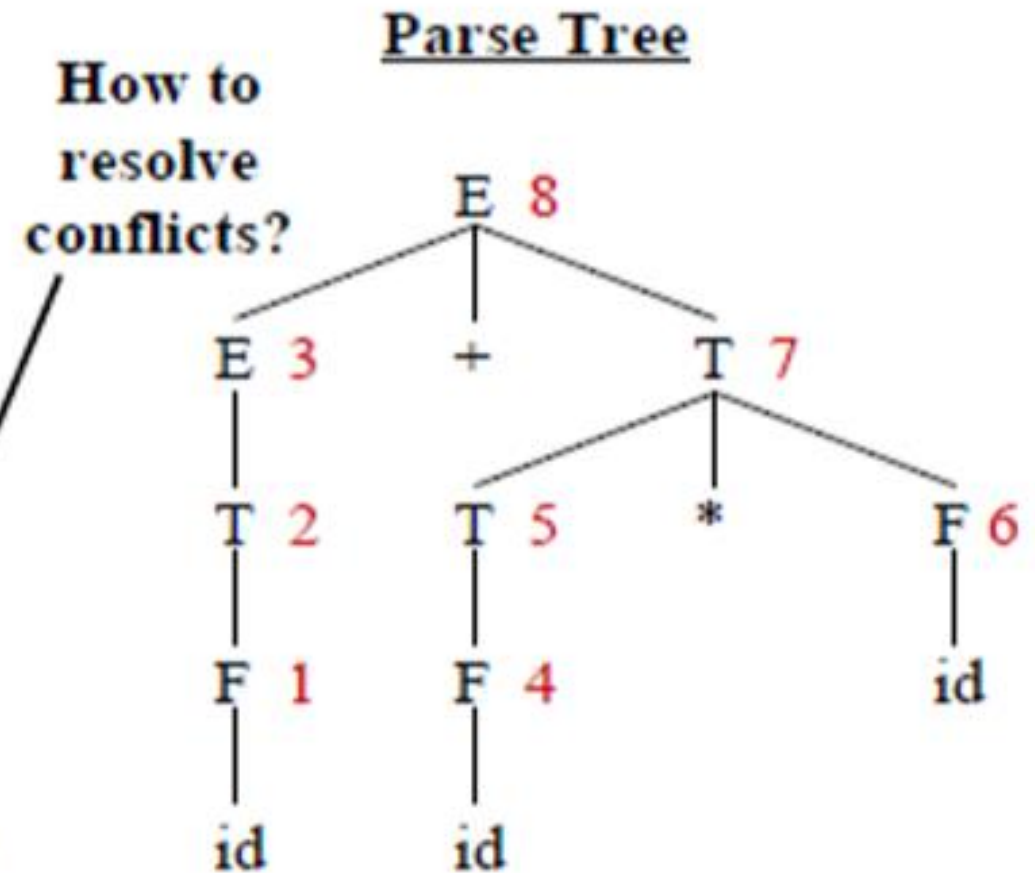
Right Sentential Form	Handle	Reducing Production
id1 * id2	id1	$F \rightarrow \text{id}$
F * id2	F	$T \rightarrow F$
T * id2	id2	$F \rightarrow \text{id}$
T * F	T * F	$E \rightarrow T * F$

Stack Implementation of Shift-Reduce Parser

The stack holds the grammar symbols and input buffer holds the string w to be parsed.

- 1. Initially stack contains only the sentinel \$, and input buffer contains the input string w .*
- 2. While stack not equal to $\$S$ or not error and input not \$ do*
 - (a) While there is no handle at the top of stack, do shift input buffer and push the symbol onto stack*
 - (b) If there is a handle on top of stack, then pop the handle and reduce the handle with its non-terminal and push it onto stack*
- 3. Done*

<u>Stack</u>	<u>Input</u>	<u>Action</u>
\$	id+id*id\$	shift
\$id	+id*id\$	reduce by $F \rightarrow id$
\$F	+id*id\$	reduce by $T \rightarrow F$
\$T	+id*id\$	reduce by $E \rightarrow T$
\$E	+id*id\$	shift
\$E+	id*id\$	shift
\$E+id	*id\$	reduce by $F \rightarrow id$
\$E+F	*id\$	reduce by $T \rightarrow F$
\$E+T	*id\$	shift (or reduce?)
\$E+T*	id\$	shift
\$E+T*id	\$	reduce by $F \rightarrow id$
\$E+T*F	\$	reduce by $T \rightarrow T*F$
\$E+T	\$	reduce by $E \rightarrow E+T$
\$E	\$	accept



Conflicts in Shift-Reduce Parsing

Some grammars cannot be parsed using shift-reduce parsing and result in conflicts. There are two kinds of shift-reduce conflicts:

1. **shift/reduce conflict:** Here, the parser is not able to decide whether to shift or to reduce.

Example: $A \rightarrow ab \mid abcd$

the stack contains \$ab, and

the input buffer contains cd\$, the parser cannot decide whether to reduce \$ab to \$A or to shift two more symbols before reducing.

Example

Consider the grammar:

$E \rightarrow E + E \mid E * E \mid id$

and input $id + id * id$

Stack	Input	Action	Stack	Input	Action
\$ E+E	*id \$	Reduce by $E \rightarrow E + E$	\$E+E	*id \$	Shift
\$ E	*id \$	Shift	\$E+E*	id \$	Shift
\$ E*	id \$	Shift	\$E+E*id	\$	Reduce by $E \rightarrow id$
\$ E*id	\$	Reduce by $E \rightarrow id$	\$E+E*E	\$	Reduce by $E \rightarrow E * E$
\$ E*E	\$	Reduce by $E \rightarrow E * E$	\$E+E	\$	Reduce by $E \rightarrow E * E$
\$ E			\$E		

2. reduce/reduce conflict:

Here, the parser cannot decide which sentential form to use for reduction.

For example $A \rightarrow bc$

$B \rightarrow abc$

and the stack contains $\$abc$, the parser cannot decide whether to reduce it to $\$aA$ or to $\$B$.

Example:

Consider the grammar:

$M \rightarrow R+R \mid R+c \mid R$

$R \rightarrow c$ and input $c+c$

Stack	Input	Action	Stack	Input	Action
\$	c+c \$	Shift	\$	c+c \$	Shift
\$ c	+c \$	Reduce by $R \rightarrow c$	\$ c	+c \$	Reduce by $R \rightarrow c$
\$ R	+c \$	Shift	\$ R	+c \$	Shift
\$ R+	c \$	Shift	\$ R+	c \$	Shift
\$ R+c	\$	Reduce by $R \rightarrow c$	\$ R+c	\$	Reduce by $M \rightarrow R+c$
\$ R+R	\$	Reduce by $M \rightarrow R+R$	\$ M	\$	
\$ M	\$				

Assignment

1. Let's take a grammar :

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow (E) \mid id$$

*Perform the Stack Implementation of Shift-Reduce Parser for
 $w = id+id*id$*

2. Explain different types of Conflict in SR parsing with examples.

LR PARSERS

- An efficient bottom-up syntax analysis technique that can be used to parse a large class of CFG is called LR(k) parsing.
- The 'L' is for left-to-right scanning of the input,
- The 'R' for constructing a rightmost derivation in reverse, and
- The 'k' for the number of input symbols.
- When 'k' is omitted, it is assumed to be 1.

Advantages of LR parsing:

- It recognizes virtually all programming language constructs for which CFG can be written.
- It is an efficient non-backtracking shift-reduce parsing method.
- A grammar that can be parsed using LR method is a proper superset of a grammar that can be parsed with predictive parser.
- It detects a syntactic error as soon as possible.

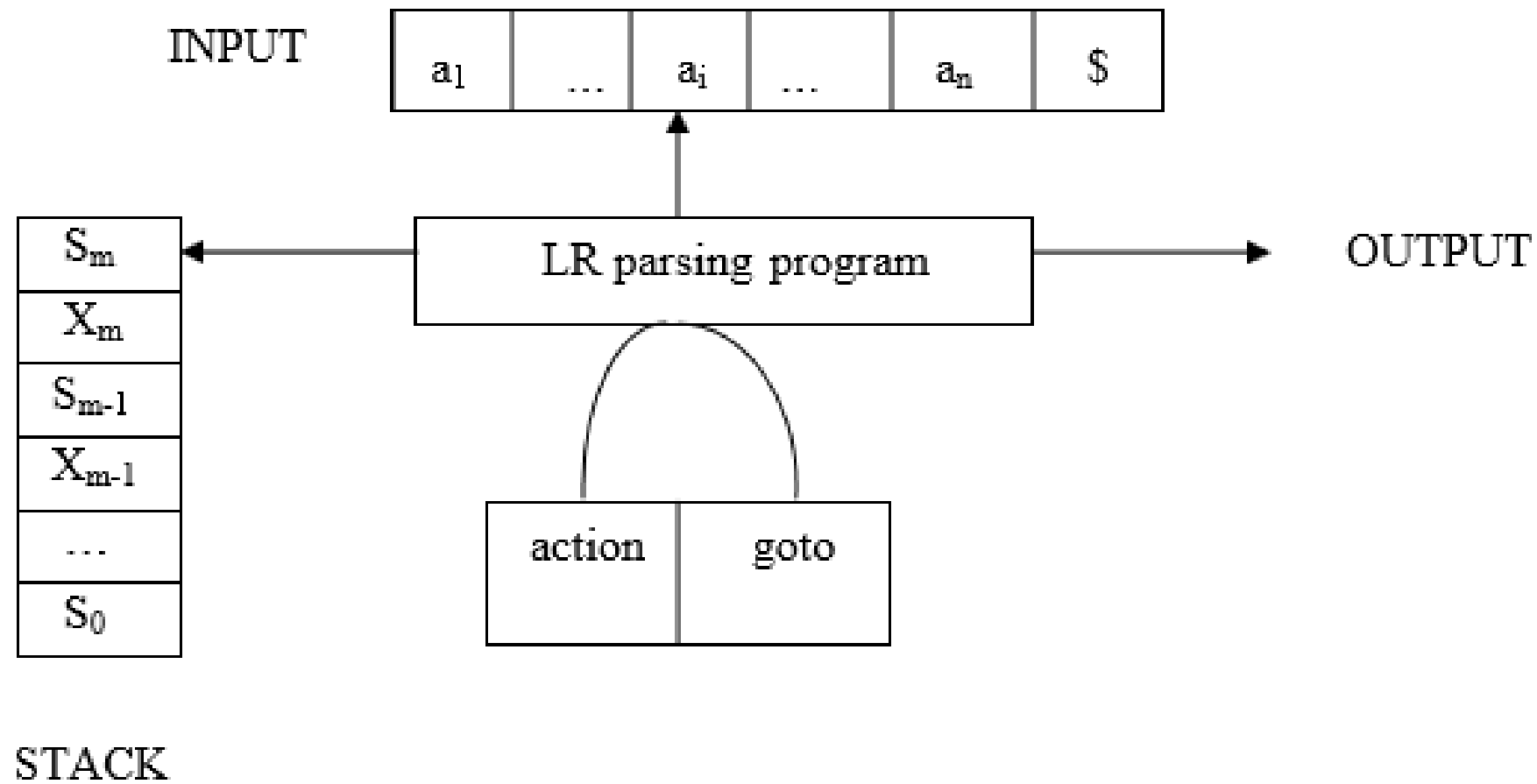
Drawbacks of LR method:

- It is too much of work to construct a LR parser by hand for a programming language grammar.
- A specialized tool, called a LR parser generator, is needed. Example: YACC

Types of LR parsing method:

1. **SLR**-Simple LR → Easiest to implement, least powerful.
2. **CLR**-Canonical LR → Most powerful, most expensive.
3. **LALR**- Look-Ahead LR → Intermediate in size and cost between the other two methods.

LR Parsers: General Structure



Constructing SLR Parsing Tables

- For constructing a **SLR parsing table** of given grammar we need to construct the **canonical LR(0) collection** of the grammar, which uses the **'closure'** operation and **'goto'** operation.

Example

Given Grammar:

$C \rightarrow AB$

$A \rightarrow a$

$B \rightarrow a$

Now, augmenting above grammar:

$C' \rightarrow C$

$C \rightarrow AB$

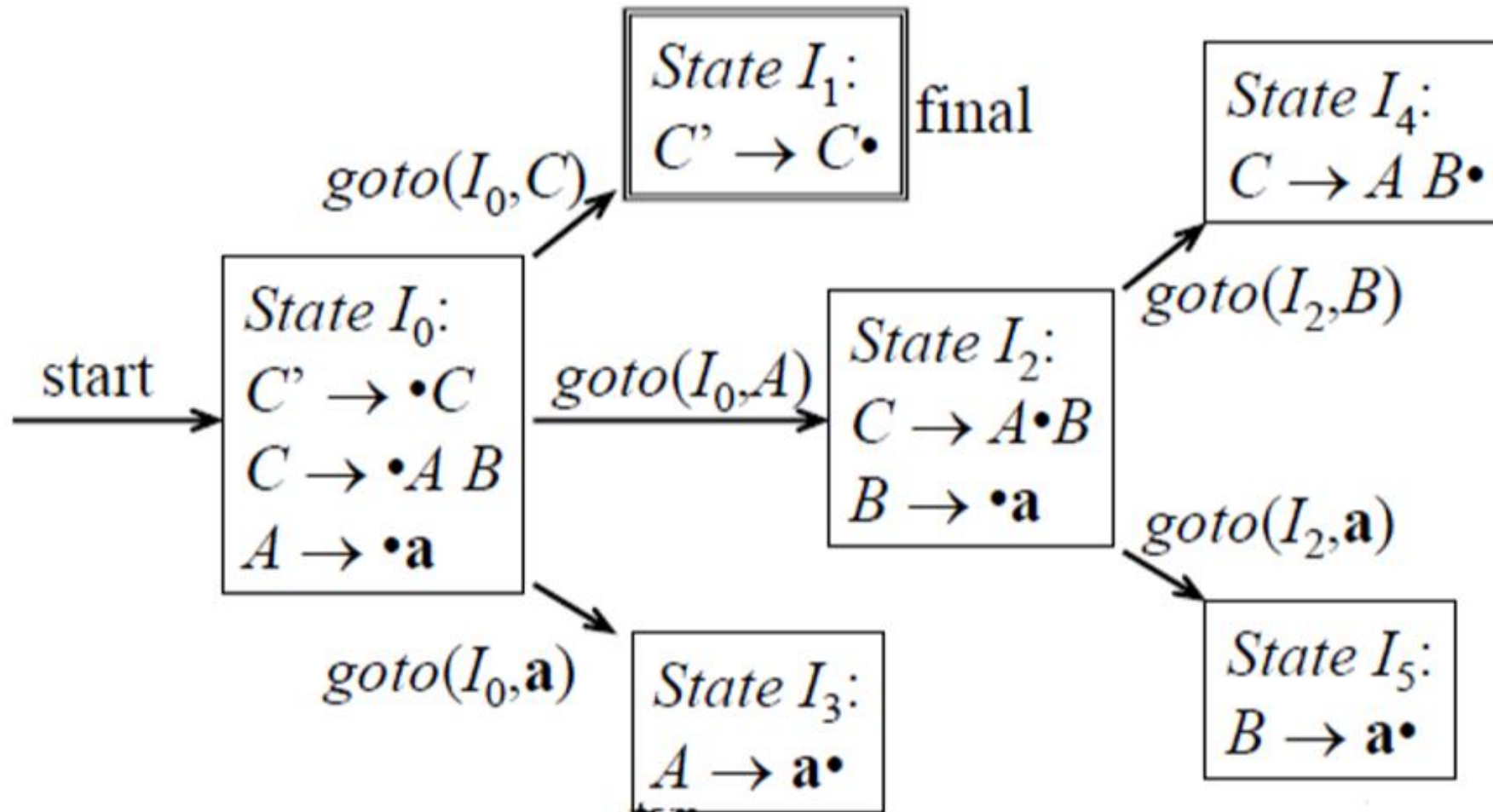
$A \rightarrow a$

$B \rightarrow a$

$I_0 = \text{closure}(C' \rightarrow \bullet C)$

$I_1 = \text{goto}(I_0, C) = \text{closure}(C' \rightarrow C \bullet)$

and so on



Example :

Find canonical LR (0) collection for the following grammar:

$E' \rightarrow E$

$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow (E) \mid id$

Find LR (0) items

$I_0 : E' \rightarrow . E$
 $E \rightarrow . E + T$
 $E \rightarrow . T$
 $T \rightarrow . T * F$
 $T \rightarrow . F$
 $F \rightarrow . (E)$
 $F \rightarrow . id$

$GOTO(I_0, E)$
 $I_1 : E' \rightarrow E .$
 $E \rightarrow E . + T$

$GOTO(I_0, T)$
 $I_2 : E \rightarrow T .$
 $T \rightarrow T . * F$

$GOTO(I_0, F)$
 $I_3 : T \rightarrow F .$

$GOTO(I_0, ($
 $I_4 : F \rightarrow (. E)$
 $E \rightarrow . E + T$
 $E \rightarrow . T$
 $T \rightarrow . T * F$
 $T \rightarrow . F$
 $F \rightarrow . (E)$
 $F \rightarrow . id$

GOTO (I₀ , id)

I₅ : F → id .

GOTO (I₁ , +)

I₆ : E → E + . T

T → . T * F

T → . F

F → . (E)

F → . id

GOTO (I₂ , *)

I₇ : T → T * . F

F → . (E)

F → . id

GOTO (I₄ , E)

I₈ : F → (E .)

E → E . + T

GOTO (I₄ , T)

I₂ : E → T .

T → T . * F

GOTO (I₄ , F)

I₃ : T → F .

GOTO (I₄ , ()

I₄ : F → (. E)

E → . E + T

E → . T

T → . T * F

T → . F

F → . (E)

F → id

GOTO (I₄ , id)

I₅ : F → id .

GOTO (I₆, T)
 I₉ : E → E + T .
 T → T . * F

GOTO (I₆, F)
 I₃ : T → F .

GOTO (I₆, (
 I₄ : F → (. E)

GOTO (I₆, id)
 I₅ : F → id .

GOTO (I₇, F)
 I₁₀ : T → T * F .

GOTO (I₇, (
 I₄ : F → (. E)
 E → . E + T
 E → . T
 T → . T * F
 T → . F
 F → . (E)
 F → . id

GOTO (I₇, id)
 I₅ : F → id .

GOTO (I₈,)
 I₁₁ : F → (E) .

GOTO (I₈, +)
 I₆ : E → E + . T
 T → . T * F
 T → . F
 F → . (E)
 F → . id

GOTO (I₉, *)
 I₇ : T → T * . F
 F → . (E)
 F → . id

$\text{FOLLOW (E)} = \{ \$,), + \}$

$\text{FOLLOW (T)} = \{ \$, +,), * \}$

$\text{FOLLOW (F)} = \{ *, +,), \$ \}$

Constructing SLR parsing table

Algorithm

1. Construct the canonical collection of sets of LR(0) items for G' .

$$C \leftarrow \{I_0 \dots I_n\}$$

2. Create the parsing action table as follows

- If $A \rightarrow \alpha.a\beta$ is in I_i and $\text{goto}(I_i, a) = I_j$ then set $\text{action}[i, a] = \textit{shift } j$.
- If $A \rightarrow \alpha.$ is in I_i , then set $\text{action}[i, a]$ to "*reduce* $A \rightarrow \alpha$ " for all 'a' in $\text{FOLLOW}(A)$ where $A \neq S'$.
- If $S' \rightarrow S.$ is in I_i , then $\text{action}[i, \$] = \textit{accept}$.
- If any conflicting actions generated by these rules, the grammar is not SLR(1).

3. Create the parsing goto table

- for all non-terminals A , if $\text{goto}(I_i, A) = I_j$ then $\text{goto}[i, A] = j$

4. All entries not defined by (2) and (3) are errors.

5. Initial state of the parser contains $S' \rightarrow .S$

Example 1:

Construct the SLR parsing table for the grammar:

$C \rightarrow AB$

$A \rightarrow a$

$B \rightarrow a$

Solution:

The augmented grammar of given grammar is:

1). $C' \rightarrow C$

2). $C \rightarrow AB$

3). $A \rightarrow a$

4). $B \rightarrow a$

Step 1:- construct the canonical LR(0) collection for the grammar as,

State I_0 :

closure($C' \rightarrow .C$)

$C' \rightarrow .C$

$C \rightarrow .AB$

$A \rightarrow .a$

State I_1 :

closure (goto(I_0, C))

closure($C' \rightarrow C.$)

$C' \rightarrow C.$

State I_2 :

closure (goto(I_0, A))

closure($C \rightarrow A.B$)

$C \rightarrow A.B$

$B \rightarrow .a$

State I_3 :

closure (goto(I_0, a))

closure($A \rightarrow a.$)

$A \rightarrow a.$

State I_4 :

closure (goto(I_2, B))

closure($C \rightarrow AB.$)

$C \rightarrow AB.$

State I_5 :

closure (goto(I_2, a))

closure($B \rightarrow .a$)

$B \rightarrow a.$

Step 2 : Construct SLR parsing table that contains both action and goto table as follows:

State I_0 :
 $C' \rightarrow \bullet C$
 $C \rightarrow \bullet A B$
 $A \rightarrow \bullet a$

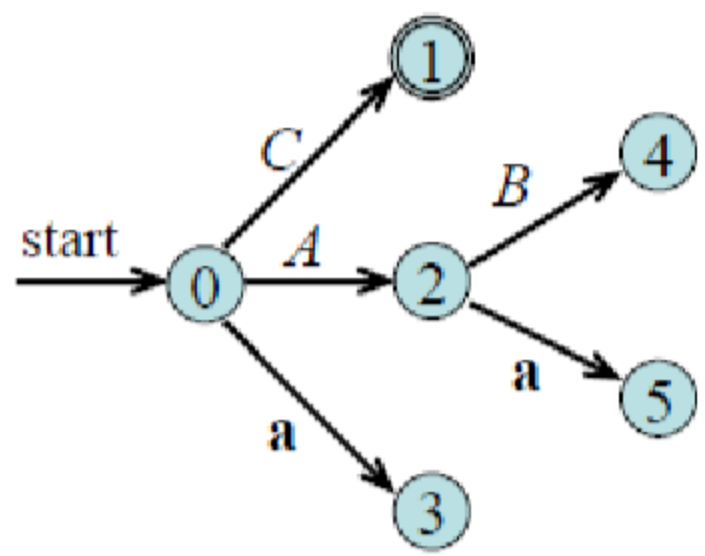
State I_1 :
 $C' \rightarrow C \bullet$

State I_2 :
 $C \rightarrow A \bullet B$
 $B \rightarrow \bullet a$

State I_3 :
 $A \rightarrow a \bullet$

State I_4 :
 $C \rightarrow A B \bullet$

State I_5 :
 $B \rightarrow a \bullet$



	a	\$	C	A	B
0	s3		1	2	
1		ac			
2	s5				4
3	r3				
4		r2			
5		r4			

Assignment-5

1. Construct the SLR parsing table for the grammar:

$E \rightarrow E + T$

$E \rightarrow T$

$T \rightarrow T * F$

$T \rightarrow F$

$F \rightarrow (E)$

$F \rightarrow id$

2. Construct the SLR parsing table for the following grammar

$X \rightarrow S S + \mid S S * \mid a$

3. Construct the SLR parsing table for the following grammar

$S' \rightarrow S$

$S \rightarrow aABe$

$A \rightarrow Abc$

$A \rightarrow b$

$B \rightarrow d$

- Example 2: Construct the SLR parsing table for the grammar:

1. $E \rightarrow E + T$

2. $E \rightarrow T$

3. $T \rightarrow T * F$

4. $T \rightarrow F$

5. $F \rightarrow (E)$

6. $F \rightarrow \text{id}$

State	ACTION						GOTO		
	id	+	*	(	)	\$	E	T	F
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

LR(1) Grammars

- SLR is so simple and can only represent the small group of grammar
LR(1) parsing uses look-ahead to avoid unnecessary conflicts in parsing table.
- LR(1) item = LR(0) item + look-ahead
- LR(0) item: $[A \rightarrow \alpha \bullet \beta]$
- LR(1) item: $[A \rightarrow \alpha \bullet \beta, a]$

Constructing LR(1) Parsing Tables

Computation of Closure for LR(1)Items:

- 1. Start with $\text{closure}(I) = I$ (where I is a set of LR(1) items)
- 2. If $[A \rightarrow \alpha \bullet B \beta, a] \in \text{closure}(I)$ then add the item $[B \rightarrow \bullet \gamma, b]$ to I if not already in I , where $b \in \text{FIRST}(\beta a)$.
- 3. Repeat 2 until no new items can be added.

Computation of Goto Operation for LR(1) Items:

- If I is a set of LR(1) items and X is a grammar symbol (terminal or non-terminal), then $\text{goto}(I, X)$ is computed as follows:
 1. For each item $[A \rightarrow \alpha \bullet X \beta, a] \in I$, add the set of items $\text{closure}(\{[A \rightarrow \alpha X \bullet \beta, a]\})$ to $\text{goto}(I, X)$ if not already there
 2. Repeat step 1 until no more items can be added to $\text{goto}(I, X)$

Construction of The Canonical LR(1) Collection:

- **Algorithm:**

Augment the grammar with production $S' \rightarrow S$

$C = \{ \text{closure}(\{S' \rightarrow .S, \$\}) \}$ (the start stat of DFA)

repeat the followings until no more set of LR(1) items can be added to C.

for each $I \in C$ and each grammar symbol $X \in (N \cup T)$

goto(I, X) $\neq \phi$ and goto(I, X) not $\in C$ then

add goto(I, X) to C

Example: Construct canonical LR(1) collection of the grammar:

$S \rightarrow AaAb$

$S \rightarrow BbBa$

$A \rightarrow \epsilon$

$B \rightarrow \epsilon$

- Its augmented grammar is:

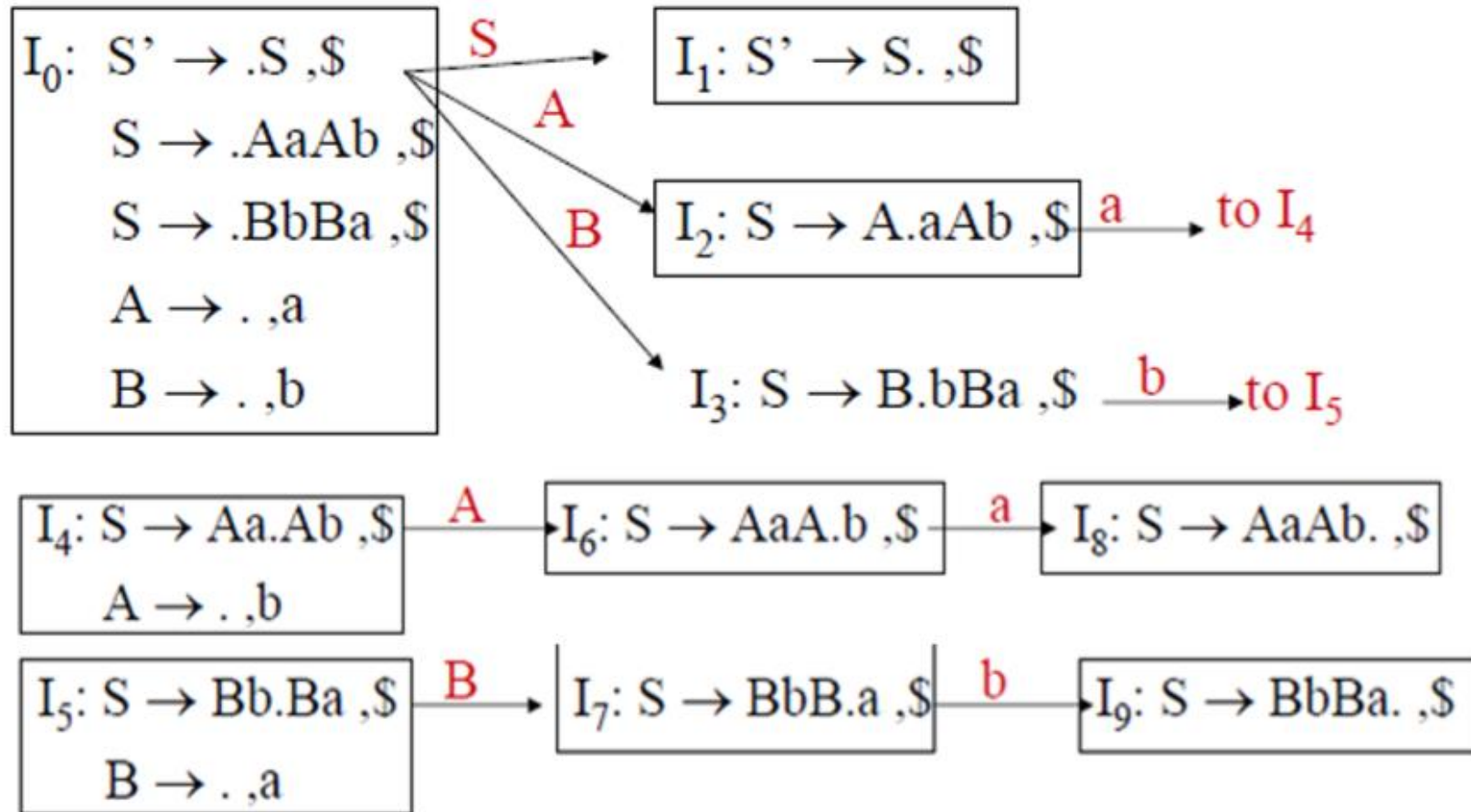
$$S' \rightarrow S$$

$$S \rightarrow AaAb$$

$$S \rightarrow BbBa$$

$$A \rightarrow \epsilon$$

$$B \rightarrow \epsilon$$



Constructing LR(1) Parsing Tables

- SLR used the LR(0) items, that is the items used were productions with an embedded dot, but contained no other (lookahead) information.
- The LR(1) items contain the same productions with embedded dots, but add a second component, which is a terminal (or \$).
- This second component becomes important only when the dot is at the extreme right.
- For LR(1) we do that reduction only if the input symbol is exactly the second component of the item.

Algorithm:

1. Construct the canonical collection of sets of LR(1) items for G' .

$$C = \{I_0 \dots I_n\}$$

2. Create the parsing action table as follows

- If $[A \rightarrow \alpha.a\beta, b]$ in I_i and $\text{goto}(I_i, a) = I_j$ then $\text{action}[i, a] = \textit{shift } j$.
- If $A \rightarrow \alpha.$, a is in I_i , then $\text{action}[i, a] = \textit{reduce } A \rightarrow \alpha$ where $A \neq S'$.
- If $S' \rightarrow S., \$$ is in I_i , then $\text{action}[i, \$] = \textit{accept}$.
- If any conflicting actions generated by these rules, the grammar is not LR(1).

3. Create the parsing goto table

- for all non-terminals A , if $\text{goto}(I_i, A) = I_j$ then $\text{goto}[i, A] = j$

4. All entries not defined by (2) and (3) are errors.

5. Initial state of the parser contains $S' \rightarrow .S, \$$

Example1: LR(1) Parsing Tables

- Construct LR(1) parsing table for given grammar:

$S' \rightarrow S$

$S \rightarrow CC$

$C \rightarrow cC$

$C \rightarrow d$

Do it yourselves.

Example 2: Construct LR(1) parsing table for the augmented grammar

1. $S' \rightarrow S$

2. $S \rightarrow L = R$

3. $S \rightarrow R$

4. $L \rightarrow * R$

5. $L \rightarrow \text{id}$

6. $R \rightarrow L$

- Step 1: At first find the canonical collection of LR(1) items of the given augmented grammar as,

State I₀:

closure($S' \rightarrow .S, \$$)

$S' \rightarrow .S, \$$

$S \rightarrow .L = R, \$$

$S \rightarrow .R, \$$

$L \rightarrow .* R, \$$

$L \rightarrow .\mathbf{Id}, =$

$R \rightarrow .L, \$$

State I₁:

closure (goto(I_0, S))

closure($S' \rightarrow S., \$$)

$S' \rightarrow S., \$$

State I₂:

closure (goto(I_0, L))

closure($(S \rightarrow L. = R, \$), (R \rightarrow L., \$)$)

$S \rightarrow L. = R, \$$

$R \rightarrow L., \$$

State I₃:

closure (goto(I_0, R))

closure($S \rightarrow R., \$$)

$S \rightarrow R., \$$

State I₄:

closure(goto($I_0, *$))

closure($L \rightarrow * .R, =$)

$\{(L \rightarrow * .R, =), (R \rightarrow .L, =), (L \rightarrow .* R, =), (L \rightarrow .\mathbf{Id}, =)\}$

State I₅ :

closure (goto(I₀, id))
 closure($L \rightarrow \mathbf{Id}.$, =)
 $L \rightarrow \mathbf{Id}.$, =

State I₆ :

closure(goto((I₂, =))
 closure($S \rightarrow L = . R$, \$)
 $S \rightarrow L = . R$, \$
 $R \rightarrow . L$, \$
 $L \rightarrow . * R$, \$
 $L \rightarrow . \mathbf{Id}$, \$

State I₇ :

closure(goto((I₄, R))
 closure($L \rightarrow^* R.$, =)
 $L \rightarrow^* R.$, =

Same Cores
 I₄ and I₁₁

State I₈ :

closure (goto(I₄, L))
 closure($R \rightarrow L.$, =)
 $R \rightarrow L.$, =

State I₉ :

closure(goto((I₆, R))
 closure($S \rightarrow L = R.$, \$)
 $S \rightarrow L = R.$, \$

State I₁₀ :

closure(goto((I₆, L))
 $R \rightarrow L.$, \$

I₅ and I₁₂

I₇ and I₁₃

State I₁₁ :

Closure(goto(I₆, *))
 Closure($L \rightarrow^* . R$, \$)
 $L \rightarrow^* . R$, \$
 $R \rightarrow . L$, \$
 $L \rightarrow . * R$, \$
 $L \rightarrow . \mathbf{id}$, \$

State I₁₂ :

closure(goto((I₆, id))
 closure($L \rightarrow \mathbf{id}.$, \$)
 $L \rightarrow \mathbf{id}.$, \$

State I₁₃ :

closure(goto((I₁₁, R))
 $L \rightarrow^* R.$, \$

I₈ and I₁₀

Step 2: Now construct LR(1) parsing table

	id	*	=	S	S	L	R
0	s5	s4			1	2	3
1				acc			
2			s6	r5			
3				r2			
4	s5	s4				8	7
5			r4	r4			
6	s12	s11				10	9
7			r3	r3			
8			r5	r5			
9				r1			
10				r5			
11	s12	s11				10	13
12				r4			
13				r3			

Same Cores

I_4 and I_{11}

I_5 and I_{12}

I_7 and I_{13}

I_8 and I_{10}

LALR(1) Grammars

- It is an intermediate grammar between the SLR and LR(1) grammar.
- A typical programming language generates thousands of states for canonical LR parsers while they generate only hundreds of states for LALR parser.
- LALR(1) parser combines two or more LR(1) sets(whose core parts are same) into a single state to reduce the table size.

Example:

$I_1: L \rightarrow \text{id.}, =$

$I_2: L \rightarrow \text{id.}, \$$



$I_{12}: L \rightarrow \text{id.}, =$

$L \rightarrow \text{id.}, \$$

Constructing LALR Parsing Tables

1. Create the canonical LR(1) collection of the sets of LR(1) items for the given grammar $C = \{I_0, \dots, I_n\}$.

2. Find each core; find all sets having that same core; replace those sets having same cores with a single set which is their union.

$C = \{I_0, \dots, I_n\}$ then $C' = \{J_1, \dots, J_m\}$ where $m \leq n$

3. Create the parsing tables (action and goto tables) same as the construction of the parsing tables of LR(1) parser.

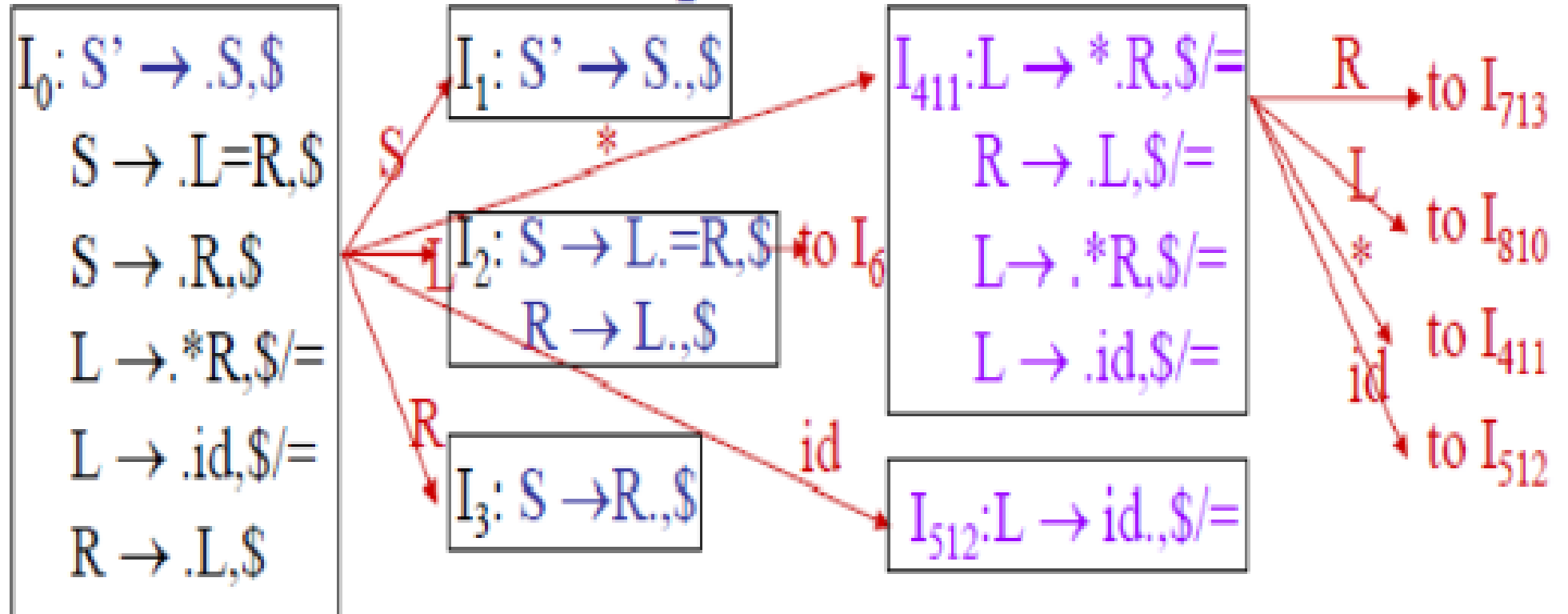
- Note that: If $J = I_1 \cup \dots \cup I_k$ since $I_1, \dots, I_k$ have same cores then cores of $\text{goto}(I_1, X), \dots, \text{goto}(I_k, X)$ must be same.
- So, $\text{goto}(J, X) = K$ where K is the union of all sets of items having same cores as $\text{goto}(I_1, X)$.

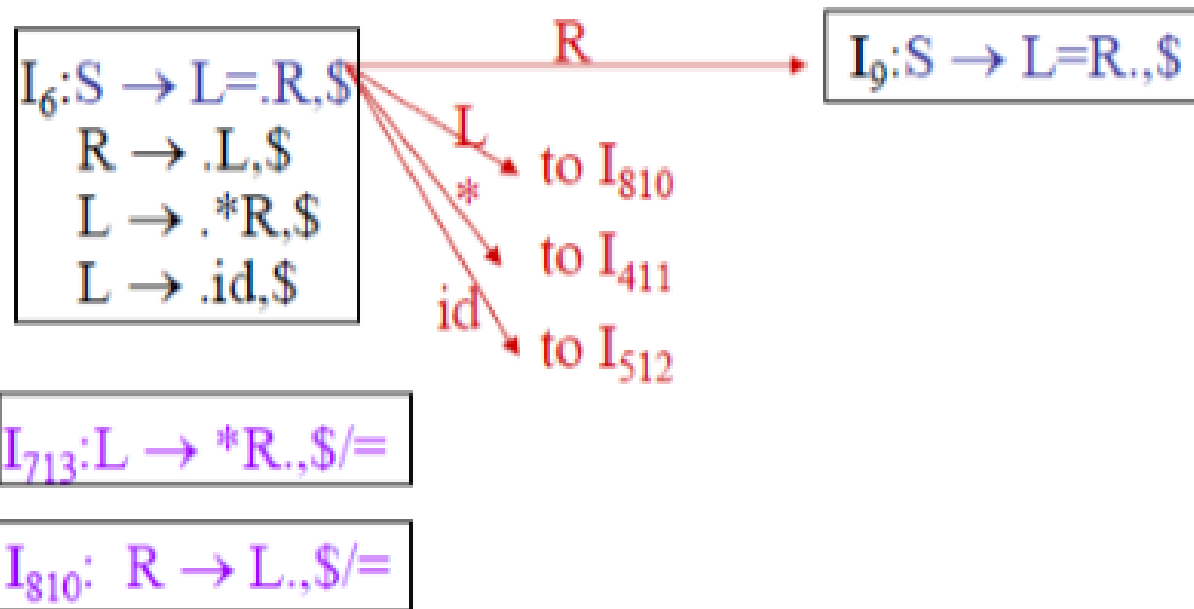
4. If no conflict is introduced, the grammar is LALR(1) grammar.

(We may only introduce reduce/reduce conflicts; we cannot introduce a shift/reduce conflict)

Example

0. $S' \rightarrow S$
1. $S \rightarrow L=R$
2. $S \rightarrow R$
3. $L \rightarrow *R$
4. $L \rightarrow id$
5. $R \rightarrow L$





Same Cores
 I_4 and I_{11}

I_5 and I_{12}

I_7 and I_{13}

I_8 and I_{10}

	id	*	=	S	S	L	R
0	s5	s4			1	2	3
1				acc			
2			s6	r5			
3				r2			
4	s5	s4				8	7
5			r4	r4			
6	s12	s11				10	9
7			r3	r3			
8			r5	r5			
9				r1			

Kernel item and Non- Kernel item

- Take any one grammar then find canonical collection of LR(0) items.
- **Kernel item:** This includes the initial items, $S' \rightarrow S$ and all items whose dot are not at the left end.
- **Non-Kernel item:** The productions of a grammar which have their dots at the left end are non-kernel items.

Limitations of Syntax Analyzers

- Syntax analyzers receive their inputs, in the form of tokens, from lexical analyzers. Lexical analyzers are responsible for the validity of a token supplied by the syntax analyzer. Syntax analyzers have the following drawbacks –
 - it cannot determine if a token is valid,
 - it cannot determine if a token is declared before it is being used,
 - it cannot determine if a token is initialized before it is being used,
 - it cannot determine if an operation performed on a token type is valid or not.

These tasks are accomplished by the semantic analyzer, which we shall study in Semantic Analysis.